

A Practical Guide to Momentum optimization

TOPIC -- MOMENTUM OPTIMIZATION

-- 01 --

$$v_w(t) := \beta^2 v_w(t-1) + (1 - \beta^2) (\nabla_w L(t-1))^2 \quad \{ \displaystyle v_w^{(t)} := \beta^2 v_w^{(t-1)} + (1 - \beta^2) \left(\nabla_w L^{(t-1)} \right)^2 \}$$

-- 02 --

Implicit updates (ISGD) As mentioned earlier, classical stochastic gradient descent is generally sensitive to learning rate η . Fast convergence requires large learning rates but this may induce numerical instability. The problem can be largely solved by considering implicit updates whereby the stochastic gradient is evaluated at the next iterate rather than the current one:

-- 03 --

$$\max_{k=0, \dots, \lfloor T/\eta \rfloor} |E[g(w_k)] - g(W_{k\eta})| \leq C\eta, \quad \{ \displaystyle \max_{k=0, \dots, \lfloor T/\eta \rfloor} |E[g(w_k)] - g(W_{k\eta})| \leq C\eta, \}$$

-- 04 --

The step size is denoted by η $\{ \displaystyle \eta \}$ (sometimes called the learning rate in machine learning) and here " $:=$ " $\{ \displaystyle := \}$ denotes the update of a variable in the algorithm. In many cases, the summand functions have a simple form that enables inexpensive evaluations of the sum-function and the sum gradient. For example, in statistics, one-parameter exponential families allow economical function-evaluations and gradient-evaluations. However, in other cases, evaluating the sum-gradient may require expensive evaluations of the gradients from all summand functions. When the training set is enormous and no simple formulas exist, evaluating the sums of gradients becomes very expensive, because evaluating the gradient requires evaluating all the summand functions' gradients. To economize on the computational cost at every iteration, stochastic gradient descent samples a subset of summand functions at every step. This is very effective in the case of large-scale machine learning problems.

-- 05 --

$$w(t) := w(t-1) - \eta m^w(t) v^w(t) + \varepsilon \sqrt{\frac{\hat{m}_w(t)}{\hat{v}_w(t)}} + \text{varepsilonpsilon}$$

-- 06 --

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next update as a linear combination of the gradient and the previous update:

-- 07 --

Stochastic gradient descent (often abbreviated SGD) is an iterative method for optimizing an objective function with suitable smoothness properties (e.g. differentiable or subdifferentiable). It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). Especially in high-dimensional optimization problems this reduces the very high computational burden, achieving faster iterations in exchange for a lower convergence rate. The basic idea behind stochastic approximation can be traced back to the Robbins–Monro algorithm of the 1950s. Today, stochastic gradient descent has become an important optimization method in machine learning.

-- 08 --

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients Q_i are sufficiently smooth. Let $T > 0$ and $g : \mathbb{R}^d \rightarrow \mathbb{R}$ be a sufficiently smooth test function. Then, there exists a constant $C > 0$ such that for all $\eta > 0$

-- 09 --

$$w^{\text{new}} := \arg \min_w \left\{ Q_i(w) + \frac{1}{2\eta} \|w - w^{\text{old}}\|^2 \right\}.$$

-- 10 --

where E denotes taking the expectation with respect to the random choice of indices in the stochastic gradient descent scheme. Since this approximation does not capture the random fluctuations around the mean behavior of stochastic gradient descent solutions to stochastic differential equations (SDEs) have been proposed as limiting objects. More precisely, the solution to the SDE

-- 11 --

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$. When optimization is done, this averaged parameter vector takes the place of w .

-- 12 --

Further reading Bottou, Léon (2004), "Stochastic Learning", Advanced Lectures on Machine Learning, LNAI, vol. 3176, Springer, pp. 146–168, ISBN 978-3-540-23122-6 Buduma, Nikhil; Locascio, Nicholas (2017), "Beyond Gradient Descent", Fundamentals of Deep Learning : Designing Next-Generation Machine Intelligence Algorithms, O'Reilly, ISBN 9781491925584 LeCun, Yann A.; Bottou, Léon; Orr, Genevieve B.; Müller, Klaus-Robert (2012), "Efficient BackProp", Neural Networks: Tricks of the Trade, Springer, pp. 9–48, ISBN 978-3-642-35288-1 Spall, James C. (2003), Introduction to Stochastic Search and Optimization, Wiley, ISBN 978-0-471-33052-3

-- 13 --

where i is uniformly sampled between 1 and n . Although theoretical convergence of this procedure happens under relatively mild assumptions, in practice the procedure can be quite unstable. In particular, when η is misspecified so that $I - \eta x_i x_i'$ has large absolute eigenvalues with high probability, the procedure may diverge numerically within a few iterations. In contrast, implicit stochastic gradient descent (shortened as ISGD) can be solved in closed-form as:

-- 14 --

where, γ is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

-- 15 --

As the algorithm sweeps through the training set, it performs the above update for each training sample. Several passes can be made over the training set until the algorithm converges. If this is done, the data can be shuffled for each pass to prevent cycles. Typical implementations may use an adaptive learning rate so that the algorithm converges. In pseudocode, stochastic gradient descent can be presented as :

-- 16 --

where the parameter w which minimizes $Q(w)$ is to be estimated, η is a step size (sometimes called the learning rate in machine learning) and α is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w , thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.